

Available Skills Reference

Generated 2026-05-25 · scripts/regen-skills-doc.py Re-run after: /plugin install, /plugin update, /plugin enable/disable, or npx skills@latest add/remove mattpocock/skills.

All skills, agents, and commands accessible in this project, organized by source. Status reflects enabledPlugins in ~/.claude/settings.json.

Invoke: /skill-name in the prompt, or they auto-trigger when conditions match. Namespaced skills use plugin:skill form (e.g. superpowers:brainstorming). Commands use /command-name. Agents are spawned by Claude Code via the Task tool.

Legend

Symbol	Meaning
	Plugin enabled — skill is available
	Plugin disabled — skill is unavailable
	superpowers (core workflow)
	mattpocock/skills (project-local .agents/skills/)
	codex (adversarial review gate)
	Other official plugins
	Built-in FleetView (no plugin required)

Invocation Types

The **Trigger** column in each table tells you who fires the skill and how:

Trigger	Who invokes	How
auto + /slash	Claude or you	Claude fires it when context matches; you can also type /skill-name explicitly
/slash only	You only	Must type /skill-name — Claude won't auto-fire (disable-model-invocation: true in SKILL.md)
auto only	Claude only	Claude auto-fires only; cannot be user-invoked (user-invocable: false in SKILL.md)
/command	You only	Explicit /command-name — commands are never auto-triggered by Claude
agent (Task tool)	Claude only	Spawned as a subagent via Task tool — not directly invocable by the user

superpowers — superpowers@claude-plugins-official v5.1.0

Core workflow skills for the gate-based dev process in docs/dev-process.md.

Name	Invoke	Trigger	Description
brainstorming	<code>superpowers:brainstorming</code>	<code>auto + /slash</code>	You MUST use this before any creative work - creating features, building components, adding functionality, or modifying behavior.
dispatching-parallel-agents	<code>superpowers:dispatching-parallel-agents</code>	<code>auto + /slash</code>	Use when facing 2+ independent tasks that can be worked on without shared state or sequential dependencies
executing-plans	<code>superpowers:executing-plans</code>	<code>auto + /slash</code>	Use when you have a written implementation plan to execute in a separate session with review checkpoints
finishing-a-development-branch	<code>superpowers:finishing-a-development-branch</code>	<code>auto + /slash</code>	Use when implementation is complete, all tests pass, and you need to decide how to integrate the work - guides completion of development work by prese...
receiving-code-review	<code>superpowers:receiving-code-review</code>	<code>auto + /slash</code>	Use when receiving code review feedback, before implementing suggestions, especially if feedback seems unclear or technically questionable - requires...
requesting-code-review	<code>superpowers:requesting-code-review</code>	<code>auto + /slash</code>	Use when completing tasks, implementing major features, or before merging to verify work meets requirements
subagent-driven-development	<code>superpowers:subagent-driven-development</code>	<code>auto + /slash</code>	Use when executing implementation plans with independent tasks in the current session
systematic-debugging	<code>superpowers:systematic-debugging</code>	<code>auto + /slash</code>	Use when encountering any bug, test failure, or unexpected behavior, before proposing fixes
test-driven-development	<code>superpowers:test-driven-development</code>	<code>auto + /slash</code>	Use when implementing any feature or bugfix, before writing implementation code
using-git-worktrees	<code>superpowers:using-git-worktrees</code>	<code>auto + /slash</code>	Use when starting feature work that needs isolation from current workspace or before executing implementation plans - ensures an isolated workspace ex...
using-superpowers	<code>superpowers:using-superpowers</code>	<code>auto + /slash</code>	Use when starting any conversation - establishes how to find and use skills, requiring Skill tool invocation before ANY response including clarifying...
verification-before-completion	<code>superpowers:verification-before-completion</code>	<code>auto + /slash</code>	Use when about to claim work is complete, fixed, or passing, before committing or creating PRs - requires running verification commands and confirming...
writing-plans	<code>superpowers:writing-plans</code>	<code>auto + /slash</code>	Use when you have a spec or requirements for a multi-step task, before touching code
writing-skills	<code>superpowers:writing-skills</code>	<code>auto + /slash</code>	Use when creating new skills, editing existing skills, or verifying skills work before deployment

mattpocock/skills — project-local .agents/skills/

Discovery-mode TDD, domain stress-testing, and lightweight workflow utilities. Installed via `npx skills@latest add mattpocock/skills` — lives in `.agents/skills/`.

Name	Invoke	Trigger	Description
caveman	caveman	auto + /slash	Ultra-compressed communication mode. Cuts token usage ~75% by dropping filler, articles, and pleasantries while keeping full technical accuracy.
diagnose	diagnose	auto + /slash	Disciplined diagnosis loop for hard bugs and performance regressions. Reproduce → minimise → hypothesise → instrument → fix → regression-test.
grill-me	grill-me	auto + /slash	Interview the user relentlessly about a plan or design until reaching shared understanding, resolving each branch of the decision tree.
grill-with-docs	grill-with-docs	auto + /slash	Grilling session that challenges your plan against the existing domain model, sharpens terminology, and updates documentation (CONTEXT).
handoff	handoff	auto + /slash	Compact the current conversation into a handoff document for another agent to pick up.
improve-codebase-architecture	improve-codebase-architecture	auto + /slash	Find deepening opportunities in a codebase, informed by the domain language in CONTEXT.md and the decisions in docs/adr/.
prototype	prototype	auto + /slash	Build a throwaway prototype to flesh out a design before committing to it.
setup-matt-pocock-skills	setup-matt-pocock-skills	/slash only	Sets up an ## Agent skills block in AGENTS.md/CLAUDE.
tdd	tdd	auto + /slash	Test-driven development with red-green-refactor loop.
to-issues	to-issues	auto + /slash	Break a plan, spec, or PRD into independently-grabbable issues on the project issue tracker using tracer-bullet vertical slices.
to-prd	to-prd	auto + /slash	Turn the current conversation context into a PRD and publish it to the project issue tracker.
triage	triage	auto + /slash	Triage issues through a state machine driven by triage roles.
write-a-skill	write-a-skill	auto + /slash	Create new agent skills with proper structure, progressive disclosure, and bundled resources.
zoom-out	zoom-out	/slash only	Tell the agent to zoom out and give broader context or a higher-level perspective.

codex — codex@openai-codex v1.0.4

Adversarial review gate (required at every phase) plus internal Codex runtime helpers. Always use **--fresh** to start a clean session.

Name	Invoke	Trigger	Description
setup	codex:setup	auto + /slash	Check whether the local Codex CLI is ready; optionally toggle the stop-time review gate.
rescue	codex:rescue	auto + /slash	Primary adversarial review entry point. Spec (Phase 1), plan (Phase 2), code (Phase 3). Always pass --fresh .

Name	Invoke	Trigger	Description
codex-cli-runtime	<code>codex:codex-cli-runtime</code>	<code>auto only</code>	Internal helper contract for calling the codex-companion runtime from Claude Code
codex-result-handling	<code>codex:codex-result-handling</code>	<code>auto only</code>	Internal guidance for presenting Codex helper output back to the user
gpt-5-4-prompting	<code>codex:gpt-5-4-prompting</code>	<code>auto only</code>	Internal guidance for composing Codex and GPT-5.4 prompts for coding, review, diagnosis, and research tasks inside the Codex Claude Code plugin

Other Plugins — Enabled

claude-code-setup — `claude-code-setup@claude-plugins-official v1.0.0`

Analyze a codebase and recommend Claude Code automations (hooks, skills, MCP servers).

Name	Invoke	Trigger	Description
claude-automation-recommender	<code>claude-code-setup:claude-automation-recommender</code>	<code>auto + /slash</code>	Analyze a codebase and recommend Claude Code automations (hooks, subagents, skills, plugins, MCP servers).

claude-md-management — `claude-md-management@claude-plugins-official v1.0.0`

Audit and improve CLAUDE.md files across the project.

Name	Invoke	Trigger	Description
claude-md-improver	<code>claude-md-management:claude-md-improver</code>	<code>auto + /slash</code>	Audit and improve CLAUDE.md files in repositories. Use when user asks to check, audit, update, improve, or fix CLAUDE.md files. Scans for all CLAUDE.
revise-claude-md	<code>/revise-claude-md</code>	<code>/command</code>	Update CLAUDE.md with learnings from this session

code-review — `code-review@claude-plugins-official vd45efab8c85b`

Automated PR review using 4 parallel agents with confidence-based scoring to filter false positives.

Name	Invoke	Trigger	Description
code-review	<code>/code-review</code>	<code>/command</code>	Code review a pull request

code-simplifier — code-simplifier@claude-plugins-official v1.0.0

Simplify recently modified code for clarity, consistency, and maintainability.

Name	Invoke	Trigger	Description
code-simplifier	code-simplifier:code-simplifier	agent (Task tool)	Simplifies and refines code for clarity, consistency, and maintainability while preserving all functionality.

commit-commands — commit-commands@claude-plugins-official vd45efab8c85b

Streamlined git workflow — commit, push, and open a PR in single commands.

Name	Invoke	Trigger	Description
clean_gone	/clean_gone	/command	Cleans up all git branches marked as [gone] (branches that have been deleted on the remote but still exist locally), including removing associated wor...
commit-push-pr	/commit-push-pr	/command	Commit, push, and open a PR
commit	/commit	/command	Create a git commit

explanatory-output-style — explanatory-output-style@claude-plugins-official v1.0.0

Educational output mode — adds **Insight** blocks with implementation rationale.

(no skills, agents, or commands found — plugin may need updating)

feature-dev — feature-dev@claude-plugins-official vd45efab8c85b

Structured 7-phase feature development workflow: explore → clarify → architect → implement → review.

Name	Invoke	Trigger	Description
feature-dev	/feature-dev	/command	Guided feature development with codebase understanding and architecture focus
code-architect	feature-dev:code-architect	agent (Task tool)	Designs feature architectures by analyzing existing codebase patterns and conventions, then providing comprehensive implementation blueprints with spe...
code-explorer	feature-dev:code-explorer	agent (Task tool)	Deeply analyzes existing codebase features by tracing execution paths, mapping architecture layers, understanding patterns and abstractions, and docum...
code-reviewer	feature-dev:code-reviewer	agent (Task tool)	Reviews code for bugs, logic errors, security vulnerabilities, code quality issues, and adherence to project conventions, using confidence-based filte...

frontend-design — frontend-design@claude-plugins-official vd45efab8c85b

Production-grade frontend UI with high design quality; avoids generic AI aesthetics.

Name	Invoke	Trigger	Description
frontend-design	frontend-design:frontend-design	auto + /slash	Create distinctive, production-grade frontend interfaces with high design quality.

hookify — hookify@claude-plugins-official vd45efab8c85b

Hook configuration management — prevent unwanted behaviors via pre/post tool hooks.

Name	Invoke	Trigger	Description
writing-hookify-rules	hookify:writing-rules	auto + /slash	This skill should be used when the user asks to “create a hookify rule”, “write a hook rule”, “configure hookify”, “add a hookify rule”, or needs guid...
configure	/configure	/command	Enable or disable hookify rules interactively
help	/help	/command	Get help with the hookify plugin
hookify	/hookify	/command	Create hooks to prevent unwanted behaviors from conversation analysis or explicit instructions
list	/list	/command	List all configured hookify rules
conversation-analyzer	hookify:conversation-analyzer	agent (Task tool)	Use this agent when analyzing conversation transcripts to find behaviors worth preventing with hooks.

playwright — playwright@claude-plugins-official vd45efab8c85b

(no skills, agents, or commands found — plugin may need updating)

pr-review-toolkit — pr-review-toolkit@claude-plugins-official vd45efab8c85b

Six specialized review agents for thorough PR analysis (comments, tests, errors, types, simplification).

Name	Invoke	Trigger	Description
review-pr	/review-pr	/command	Comprehensive PR review using specialized agents
code-reviewer	pr-review-toolkit:code-reviewer	agent (Task tool)	Use this agent when you need to review code for adherence to project guidelines, style guides, and best practices.
code-simplifier	pr-review-toolkit:code-simplifier	agent (Task tool)	Use this agent when code has been written or modified and needs to be simplified for clarity, consistency, and maintainability while preserving all fu...
comment-analyzer	pr-review-toolkit:comment-analyzer	agent (Task tool)	Use this agent when you need to analyze code comments for accuracy, completeness, and long-term maintainability.

Name	Invoke	Trigger	Description
pr-test-analyzer	pr-review-toolkit:pr-test-analyzer	agent (Task tool)	Use this agent when you need to review a pull request for test coverage quality and completeness.
silent-failure-hunter	pr-review-toolkit:silent-failure-hunter	agent (Task tool)	Use this agent when reviewing code changes in a pull request to identify silent failures, inadequate error handling, and inappropriate fallback behavi...
type-design-analyzer	pr-review-toolkit:type-design-analyzer	agent (Task tool)	Use this agent when you need expert analysis of type design in your codebase.

pyright-lsp — pyright-lsp@claude-plugins-official v1.0.0

Pyright LSP integration for Python type checking.

(no skills, agents, or commands found — plugin may need updating)

ralph-loop — ralph-loop@claude-plugins-official v1.0.0

Iterative agent loop — repeatedly feeds Claude a task until a “completion promise” string appears.

Name	Invoke	Trigger	Description
cancel-ralph	/cancel-ralph	/command	Cancel active Ralph Loop
help	/help	/command	Explain Ralph Loop plugin and available commands
ralph-loop	/ralph-loop	/command	Start Ralph Loop in current session

remember — remember@claude-plugins-official v0.7.2

Session continuity — saves state to `.remember/now.md` for clean resumption next session.

Name	Invoke	Trigger	Description
remember	remember:remember	auto + /slash	Save session state for clean continuation next session.

security-guidance — security-guidance@claude-plugins-official vd45efab8c85b

Security-focused code review and vulnerability guidance.

(no skills, agents, or commands found — plugin may need updating)

skill-creator — skill-creator@claude-plugins-official vd45efab8c85b

Create, edit, eval, and benchmark Claude Code skills.

Name	Invoke	Trigger	Description
skill-creator	skill-creator:skill-creator	auto + /slash	Create new skills, modify and improve existing skills, and measure skill performance.

supabase — supabase@claude-plugins-official v0.1.6

Postgres performance optimization and best practices from Supabase.

Name	Invoke	Trigger	Description
supabase	supabase:supabase	auto + /slash	Use when doing ANY task involving Supabase.
supabase-postgres-best-practices	supabase:supabase-postgres-best-practices	auto + /slash	Postgres performance optimization and best practices from Supabase.

typescript-lsp — typescript-lsp@claude-plugins-official v1.0.0

TypeScript LSP integration for diagnostics and in-context code intelligence.

(no skills, agents, or commands found — plugin may need updating)

Built-in FleetView Skills

Embedded in Claude Code itself — no plugin installation required.

Name	Invoke	Trigger	Description
verify	verify	auto + /slash	Run the app and observe behavior to confirm a change works end-to-end (not just tests).
run	run	auto + /slash	Launch this project's app to see a change working live.
init	init	auto + /slash	Initialize a new CLAUDE.md with codebase documentation.
code-review	code-review	auto + /slash	Review current diff for correctness bugs. Pass --comment to post as inline PR comments.
security-review	security-review	auto + /slash	Security-focused code review.
update-config	update-config	auto + /slash	Configure Claude Code via settings.json — hooks, permissions, env vars, automated behaviors.
keybindings-help	keybindings-help	auto + /slash	Customize keyboard shortcuts in ~/.claude/keybindings.json .
fewer-permission-prompts	fewer-permission-prompts	auto + /slash	Scan transcripts for common read-only calls; add prioritized allowlist to reduce prompts.
loop	loop	auto + /slash	Run a prompt or slash command on a recurring interval (e.g. /loop 5m /foo).
schedule	schedule	auto + /slash	Create/manage scheduled remote agents on a cron schedule or one-time delay.
claude-api	claude-api	auto + /slash	Build, debug, and optimize Claude API / Anthropic SDK apps with prompt caching.

Name	Invoke	Trigger	Description
statusline-setup	<i>(agent)</i>	agent (Task tool)	Configure the Claude Code status line setting.

Disabled Plugins

Installed but toggled off in `~/.claude/settings.json`. Skills are unavailable until re-enabled.

Plugin	Version	Re-enable
asana@claude-plugins-official	d45efab8c85b — Asana task and project management commands.	<code>/plugin enable asana@claude-plugins-official</code>
deploy-on-aws@claude-plugins-official	1.2.0 — Deploy to AWS and generate validated architecture diagrams (draw.io with AWS4 icons).	<code>/plugin enable deploy-on-aws@claude-plugins-official</code>
github@claude-plugins-official	d45efab8c85b — GitHub PR, issue, and repo management commands.	<code>/plugin enable github@claude-plugins-official</code>

Project-Specific Skill Routing

When multiple skills handle the same task, use this table (from `docs/plugins.md`):

Task	Condition	Skill
TDD	Behaviors known upfront	<code>superpowers:test-driven-development</code>
TDD	Behavior discovered during implementation	<code>tdd (mattpocock)</code>
Debugging	Clear feedback loop (test failure, stack trace)	<code>superpowers:systematic-debugging</code>
Debugging	Building the feedback loop <i>is</i> the hard problem	<code>diagnose (mattpocock)</code>
Session handoff	End of session	<code>remember:remember</code>
Mid-task handoff	Passing work to a subagent	<code>handoff (mattpocock)</code>
Code review	Claude review (first)	<code>superpowers:requesting-code-review</code>
Adversarial review	Every phase gate	<code>codex:rescue --fresh</code>
Domain stress-test	After brainstorming, before Codex	<code>grill-with-docs (mattpocock)</code>
Writing skills	Any skill authoring	<code>superpowers:writing-skills</code>